BioMind-BG: Complete Technical Documentation

A Biologically Faithful Basal Ganglia Simulation
File-by-File Code Reference with Biological Explanations

BioMind Project

March 2026

Contents

1 Project Overview **2**
1.1 What It Does . . . 2
1.2 File Structure . . . 2
1.3 Data Flow . . . 2

2 params.py — All Biological Constants **3**
2.1 Simulation Constants . . . 3
2.2 Default Neuron Parameters . . . 3
2.2.1 Calcium Dynamics . . . 3
2.2.2 T-Current (Rebound Bursting) . . . 4
2.2.3 Additional Channels . . . 4
2.3 Population-Specific Overrides . . . 4
2.4 Receptor Kinetics . . . 4
2.5 External Background Input . . . 5
2.6 Pathway Connectivity . . . 5
2.7 Dopamine Parameters . . . 6
2.8 Q-Learning Parameters . . . 6
2.9 Trial Parameters . . . 6

3 populations.py — Building the Network **8**
3.1 Channel Architecture . . . 8
3.2 build_population_data() . . . 8
3.3 _build_single_pop() . . . 8
3.4 build_connectivity() . . . 9
3.5 _get_population_pairs() — The syn vs all Logic . . . 9

4 agent.py — The Living Network State **10**
4.1 Core State Variables . . . 10
4.2 Parameter Expansion . . . 10
4.3 Synaptic Conductance State . . . 10
4.4 Connectivity Matrices . . . 11
4.5 Dopamine State . . . 11
4.6 Temporary Arrays and Firing Rate Buffer . . . 11

5 timestep.py — The Core Simulation **12**
5.1 Step 1: Reset Spike Indicators . . . 12
5.2 Step 2: External Input (Ornstein-Uhlenbeck) . . . 12
5.3 Step 3: Spike Propagation . . . 12
5.4 Step 4: T-Current (Rebound Bursting) . . . 12
5.5 Step 5: Membrane Voltage Update . . . 13
5.6 Step 6: Spike Detection . . . 13
5.7 Step 7: Dopamine Plasticity (Three-Factor Learning) . . . 13
5.8 Step 8: Firing Rate Buffer . . . 14

6 qlearning.py — Reward to Dopamine Bridge **15**
6.1 QLearner Class . . . 15
6.2 update() — The Core Learning Step . . . 15

7 trial.py — The Behavioral Task **16**
7.1 Phase 0: Stimulus Presentation . . . 16
7.2 Phase 1: Movement Delay . . . 16
7.3 Phase 2: Reward and Inter-Trial Interval . . . 16
7.4 Key Implementation Detail . . . 16

8 run.py — Orchestration **17**

9 Validated Results **18**
9.1 Experiment 1: Baseline Firing Rates . . . 18
9.2 Experiment 4: Lesion Studies . . . 18

1 Project Overview

BioMind-BG is a spiking neural network simulation of the cortico-basal-ganglia-thalamic (CBGT) circuit. It simulates approximately 3,000 neurons organized into 9 brain nuclei, connected by 27 biologically constrained pathways, learning from dopamine reward signals.

1.1 What It Does

1. Builds 9 populations of neurons (Cortex, D1-MSN, D2-MSN, FSI, GPe, STN, GPi, Thalamus, Cortical Interneurons)
2. Wires them together exactly as they are in a real brain
3. Simulates spiking dynamics with AMPA, GABA, and NMDA receptors
4. Detects decisions via thalamic firing rate threshold crossing
5. Learns from reward through dopamine-modulated synaptic plasticity

1.2 File Structure

File	Lines	Purpose
`params.py`	190	All biological constants with units
`populations.py`	130	Population construction and connectivity matrices
`agent.py`	160	Neural state initialization (all arrays)
`timestep.py`	260	Differential equations (the core simulation loop)
`qlearning.py`	65	Q-values and RPE-to-dopamine conversion
`trial.py`	130	Trial state machine (stimulus, decision, reward)
`run.py`	170	Main simulation runner

1.3 Data Flow

The data flows through the system in this order:

1. `params.py` defines constants
2. `populations.py` builds population dicts and connectivity matrices from those constants
3. `agent.py` expands everything into per-neuron NumPy arrays
4. `timestep.py` runs the differential equations every 0.2ms
5. `trial.py` manages the behavioral task (stimulus, decision detection, reward delivery)
6. `qlearning.py` converts reward outcomes into dopamine signals
7. `run.py` orchestrates everything

2 params.py — All Biological Constants

This file contains every number the simulation uses. No computation happens here, just definitions. Every parameter has a unit and a biological justification.

2.1 Simulation Constants

```
DT = 0.2            # ms, integration timestep
BUFFER_LEN = 300    # rolling window for firing rate
```

DT = 0.2 ms: A real spike lasts about 1–2 ms. If our timestep were larger than that, we would miss spikes entirely. 0.2 ms gives us 5–10 samples per spike, which is enough to track the voltage dynamics accurately. Smaller would be more precise but slower. This is the sweet spot that CBGTPy validated.

BUFFER_LEN = 300: The rolling buffer for measuring firing rates. 300 slots $\times$ 0.2 ms = 60 ms window. A neuron fires spikes as discrete events. To measure "firing rate" we need to count spikes over some window of time. 60 ms is long enough for a stable average but short enough to track rapid changes during a decision. The buffer is circular: when slot 300 is reached, it wraps back to slot 0.

2.2 Default Neuron Parameters

Every neuron in the model starts with these defaults. Specific populations then override what they need.

```
NEURON_DEFAULTS = {
    'N':          75,       # neurons per population per channel
    'C':          0.5,      # nF, membrane capacitance
    'Taum':       20.0,     # ms, membrane time constant
    'RestPot':    -70.0,    # mV, resting membrane potential
    'ResetPot':   -55.0,    # mV, post-spike reset potential
    'Threshold':  -50.0,    # mV, spike threshold
}
```

N = 75: Each population gets 75 neurons per action channel. Not 1, not 10,000. 75 is enough to get population-level statistics (averaging over noise) but small enough to simulate fast.

C = 0.5 nF: Capacitance. The cell membrane is a thin insulator between two conductors (inside and outside the cell). It stores charge like a capacitor. Higher C means the neuron needs more current to change its voltage. Think of it as the neuron's "inertia."

Taum = 20 ms: Membrane time constant. This is how fast voltage decays back to rest when there is no input. At 20 ms, the neuron "forgets" an input after about 20 ms. This sets the timescale of integration.

RestPot = −70 mV: Where the voltage sits when nothing is happening. This is set by the balance of leak K^+ and Na^+ channels. Every neuron in your brain right now is sitting near −70 mV.

ResetPot = −55 mV and Threshold = −50 mV: After a spike, voltage resets to −55. The next spike requires reaching −50. That is only a 5 mV gap, which means neurons can fire again quickly if input is strong enough.

2.2.1 Calcium Dynamics

```
    'RestPot_ca': -85.0,    # mV, calcium reversal potential
    'Alpha_ca':   0.5,      # uM, calcium increment per spike
    'Tau_ca':     80.0,     # ms, calcium decay time constant
    'Eff_ca':     0.0,      # calcium efficacy (unused)
```

Every time a neuron spikes, a little calcium flows in (Alpha_ca = 0.5). This calcium activates a potassium channel (AHP, afterhyperpolarization) that pulls the voltage **down**. So after a burst of spikes, calcium builds up and the neuron gets harder to fire. It is a self-braking mechanism — spike-rate adaptation. The calcium then slowly decays away with Tau_ca = 80 ms, letting the neuron recover.

2.2.2 T-Current (Rebound Bursting)

```
    'tauhm':      20.0,     # ms, burst inactivation time constant
    'tauhp':      100.0,    # ms, de-inactivation time constant
    'V_h':        -60.0,    # mV, burst activation threshold
    'V_T':        120.0,    # mV, T-current reversal potential
    'g_T':        0.0,      # mS/cm2, max conductance (OFF by default)
```

This mechanism is **OFF by default** ($g_T = 0$) but turned ON for STN and GPe ($g_T = 0.06$). When a neuron is held below -60 mV (inhibited) for a while, the h gating variable slowly charges up (de-inactivation, $\tau = 100$ ms). When the inhibition releases, that stored h fires a burst of calcium current that pushes the neuron way above threshold. That is a rebound burst.

This is how STN fires a burst when GPe releases it from inhibition — critical for the indirect pathway.

2.2.3 Additional Channels

```
    'g_adr_max':  0.0,      # Anomalous delayed rectifier (OFF)
    'g_k_max':    0.0,      # Outward rectifying K+ (OFF)
```

Both are OFF by default. Included for biological completeness and future extensions.

2.3 Population-Specific Overrides

```
POP_SPECIFIC = {
    'Cx':   {'N': 204, 'dpmn_cortex': 1},
    'CxI':  {'N': 186, 'C': 0.2, 'Taum': 10.0},
    'dSPN': {},
    'iSPN': {},
    'FSI':  {'C': 0.2, 'Taum': 10.0},
    'GPe':  {'N': 750, 'g_T': 0.06, 'Taum': 20.0},
    'STN':  {'N': 750, 'g_T': 0.06},
    'GPi':  {},
    'Th':   {'Taum': 27.78},
}
```

2.4 Receptor Kinetics

```
RECEPTOR_DEFAULTS = {
    'Tau_AMPA':     2.0,    # ms
    'RevPot_AMPA':  0.0,    # mV
    'Tau_GABA':     5.0,    # ms
```

Table 1: Population overrides and biological justification.

Population	Overrides	Why
Cx (Cortex)	N=204, dpmn_cortex=1	Largest pop. Flag for plasticity source
CxI (Cortical Interneurons)	C=0.2, Taum=10	PV+ cells: small, fast
dSPN (D1-MSN)	{} (all defaults)	Quiet neurons, need strong input
iSPN (D2-MSN)	{} (all defaults)	Same as dSPN
FSI (Fast-Spiking)	C=0.2, Taum=10	PV+ interneurons: small, fast
GPe	N=750, g_T=0.06	Pacemaker, rebound bursting ON
STN	N=750, g_T=0.06	Rebound bursting ON
GPi	{} (all defaults)	Output gate
Th (Thalamus)	Taum=27.78	Slower integration, smooths GPi input

```
    'RevPot_GABA':   -70.0,   # mV
    'Tau_NMDA':      100.0,   # ms
    'RevPot_NMDA':   0.0,     # mV
}
NMDA_ALPHA = 0.6332
```

AMPA ($\tau = 2$ ms, $E_{rev} = 0$ mV): Fast excitatory. Opens immediately on glutamate binding, lets Na^+ in, pushes voltage toward 0 mV. The only plastic receptor — all learning happens at AMPA synapses.

GABA$_A$ ($\tau = 5$ ms, $E_{rev} = -70$ mV): Fast inhibitory. Opens Cl^- channels, clamps voltage at -70 mV (rest). This is how dSPN inhibits GPi, how GPi inhibits Thalamus, how GPe inhibits STN.

NMDA ($\tau = 100$ ms, $E_{rev} = 0$ mV): Slow excitatory. Same reversal as AMPA but 50× slower. A single spike's effect lingers for 100 ms. Has two special properties: Mg^{2+} block (only opens when neuron is already depolarized) and saturation (NMDA_ALPHA prevents unbounded accumulation).

2.5 External Background Input

```
BASESTIM = {
    'Cx':    {'FreqExt_AMPA': 2.3,  'MeanExtEff_AMPA': 2.0,
              'MeanExtCon_AMPA': 800},
    'GPe':   {'FreqExt_AMPA': 4.0,  'MeanExtEff_AMPA': 2.0,
              'MeanExtCon_AMPA': 800,
              'FreqExt_GABA': 2.0,  'MeanExtEff_GABA': 2.0,
              'MeanExtCon_GABA': 2000},
    ...
}
```

Every nucleus is constantly bombarded by thousands of synapses from brain regions we are not modeling. Three numbers per population work together: **FreqExt** (how fast external neurons fire), **MeanExtEff** (how strong each connection is), **MeanExtCon** (how many external connections). The total background drive is roughly Freq × Eff × N.

Key observation: **GPe is the only population with external GABA input** (2000 connections). This represents striatal inhibition from brain regions outside our model.

2.6 Pathway Connectivity

The complete wiring diagram. Each row: (source, destination, receptor, type, connection probability, efficacy, plastic).

Type is critical: `'syn'` means channel-specific (action 1 cortex only connects to action 1 striatum). `'all'` means all-to-all across channels (STN excites ALL GPi channels).

Table 2: Key pathways in the CBGT circuit.

Source	Dest	Receptor	Type	Prob	Eff	Plastic
Direct Pathway (Go)						
Cx	dSPN	AMPA	syn	1.0	0.015	**Yes**
dSPN	GPi	GABA	syn	1.0	2.09	No
Indirect Pathway (NoGo)						
Cx	iSPN	AMPA	syn	1.0	0.015	**Yes**
iSPN	GPe	GABA	syn	1.0	4.07	No
GPe	STN	GABA	syn	0.067	0.35	No
STN	GPi	NMDA	all	1.0	0.038	No
Output						
GPi	Th	GABA	syn	1.0	0.3315	No

Only **two** connections are plastic: Cx→dSPN AMPA and Cx→iSPN AMPA. All learning in this model happens at the corticostriatal synapse, modulated by dopamine.

2.7 Dopamine Parameters

```
D1_PARAMS = {
    'dpmn_type':     1,
    'dpmn_alphaw':   39.5,      # positive = LTP on reward
    'dpmn_wmax':     0.055,
}
D2_PARAMS = {
    'dpmn_type':     2,
    'dpmn_alphaw':   -38.2,     # negative = LTD on reward
    'dpmn_wmax':     0.035,
}
```

The sign of `alphaw` is the entire Go/NoGo mechanism. When dopamine fires (reward): D1 gets $+39.5\times$ positive fDA = LTP (Go strengthened). D2 gets $-38.2\times$ positive fDA = LTD (NoGo weakened). On punishment, everything reverses.

2.8 Q-Learning Parameters

```
Q_PARAMS = {
    'q_alpha':   0.1,     # 10% update per trial
    'C_scale':   80.0,    # RPE amplification for neural scale
    'Q_init':    0.5,     # no prior bias
}
```

`C_scale = 80` converts an abstract prediction error (small number between -2 and $+1$) into a neural-scale dopamine signal. Without this scaling, the RPE would produce negligible weight changes in the spiking network.

2.9 Trial Parameters

```
TRIAL_DEFAULTS = {
    'thalamic_threshold':   30.0,     # Hz
    'choice_timeout':       1000,     # ms
```

```
    'movement_time':        300,      # ms
    'inter_trial_interval': 300,      # ms
    'max_stim':             3.0,
    'warmup_steps':         5000,
}
```

thalamic_threshold = 30 Hz: A decision happens when any thalamic channel exceeds 30 Hz. At baseline, thalamus fires at ∼7 Hz. It must roughly quadruple. This only happens when dSPN successfully inhibits GPi enough to release thalamus.

movement_time = 300 ms: Post-decision delay before reward. The eligibility trace ($\tau_E = 100$ ms) must survive this delay for learning to work.

max_stim = 3.0: During stimulus, cortex gets baseline + 3.0 kHz, driving it from ∼2.3 to 5.3 kHz, producing ∼116 Hz firing.

3 populations.py — Building the Network

This file answers two questions: (1) How do we create actual populations from the parameter templates? (2) How do we build the weight matrices connecting them?

3.1 Channel Architecture

For a 2-choice task, some populations are **channel-specific** (each action gets its own copy) and some are **shared** (one copy serves all actions).

- **Channel-specific:** GPi, STN, GPe, dSPN, iSPN, Cx, Th. Each action needs its own set. Action 0's cortex does not leak into action 1's striatum.
- **Shared:** FSI, CxI. Interneurons that provide blanket inhibition across all channels.

For 2 actions: $7 \times 2 + 2 = 16$ populations total.

3.2 build_population_data()

```
def build_population_data(n_actions=2):
    channel_pops = ['GPi','STN','GPe','dSPN','iSPN','Cx','Th']
    shared_pops = ['FSI', 'CxI']
    pops = []
    pop_index = {}
    for name in POP_NAMES:
        if name in channel_pops:
            for ch in range(n_actions):
                pop = _build_single_pop(name, ch)
                idx = len(pops)
                pops.append(pop)
                pop_index[(name, ch)] = idx
        else:
            pop = _build_single_pop(name, -1)
            idx = len(pops)
            pops.append(pop)
            pop_index[(name, -1)] = idx
    return pops, pop_index
```

`pop_index` is a lookup dictionary. To find "where is dSPN for action 0?" you ask `pop_index[('dSPN', 0)]` and get back the array index. Shared populations use channel $= -1$.

3.3 _build_single_pop()

Assembles a complete parameter dictionary by layering:

1. Start with NEURON_DEFAULTS (the template)
2. Apply POP_SPECIFIC overrides (e.g., GPe gets $g_T = 0.06$)
3. Add RECEPTOR_DEFAULTS (AMPA/GABA/NMDA kinetics)
4. Add BASESTIM (background bombardment)
5. Fill missing stim params with 0 (prevents KeyError)
6. Add DPMN params only for dSPN and iSPN

3.4 build_connectivity()

Turns the PATHWAYS table into actual NumPy weight matrices.

```
def build_connectivity(pops, pop_index, n_actions=2):
    for receptor in ['AMPA', 'GABA', 'NMDA']:
        for pathway in PATHWAYS:
            if rec != receptor: continue
            pairs = _get_population_pairs(...)
            for src_idx, dest_idx in pairs:
                if prob >= 1.0:
                    con_data = np.ones((n_src, n_dest))
                else:
                    con_data = (np.random.rand(n_src,n_dest) < prob)
                eff_data = con_data * eff
```

For each population pair: if connection probability is 1.0, all synapses exist. If less than 1.0, each possible synapse gets a random draw. **Efficacy** = **connection** × **weight**. If a synapse does not exist (con = 0), its efficacy is automatically 0.

3.5 _get_population_pairs() — The syn vs all Logic

`'syn'`: Only matching channels connect. Cx channel 0 → dSPN channel 0. No cross-talk. The condition is `src_ch == dest_ch` (or either is −1 for shared populations).

`'all'`: Every source connects to every destination regardless of channel. This is how STN provides a broad brake signal to all GPi channels simultaneously.

4 agent.py — The Living Network State

`params.py` is the blueprint. `populations.py` is the construction plan. `agent.py` is the actual building with all its rooms and plumbing ready to run.

Every attribute is a list of NumPy arrays indexed by population ID. So `self.V[3]` is the voltage array for population 3, and `self.V[3][47]` is the voltage of neuron 47 in that population.

4.1 Core State Variables

```
self.V = [np.ones(p['N']) * p['RestPot'] for p in pops]
self.Ca = [np.zeros(p['N']) for p in pops]
self.h = [np.ones(p['N']) * p['h'] for p in pops]
self.RefrState = [np.zeros(p['N'], dtype=int) for p in pops]
self.spikes = [[] for _ in pops]
self.timesincelastspike = [np.zeros(p['N']) for p in pops]
self.Ptimesincelastspike = [np.zeros(p['N']) for p in pops]
```

- **V** starts at -70 mV for every neuron. Resting state.
- **Ca** starts at 0. Calcium only accumulates when neurons spike.
- **h** starts at 1.0 (fully de-inactivated). STN and GPe are ready to rebound burst immediately.
- **RefrState** starts at 0. When a neuron spikes, set to 10 (= 2 ms refractory).
- **spikes** is rebuilt every timestep. Contains indices of neurons that fired.
- **Ptimesincelastspike** saves the timer value BEFORE reset, needed for NMDA saturation.

4.2 Parameter Expansion

Every scalar parameter gets expanded to a per-neuron array using `np.full()`:

```
self.Taum = [np.full(p['N'], p['Taum']) for p in pops]
```

This creates an array of 75 identical values. Why not use the scalar directly? Because the timestep code does `a.V[i] += ... / a.Taum[i]` where `V[i]` is a 75-element array. Keeping everything as arrays is consistent and enables future per-neuron heterogeneity.

4.3 Synaptic Conductance State

Two sources of synaptic input per receptor:

- **ExtS** (external): Background noise following Ornstein-Uhlenbeck process. Fluctuates around **ExtMuS** with standard deviation **ExtSigmaS**.
- **LS** (lateral): Driven by spikes from other populations in the network. When neuron j in GPe spikes, it adds to `LS_GABA` of connected STN neurons.

4.4 Connectivity Matrices

Unpacked from the connectivity dict built by `populations.py`:

```
self.AMPA_con, self.AMPA_eff, self.AMPA_plastic = connectivity['AMPA']
self.GABA_con, self.GABA_eff, _ = connectivity['GABA']
self.NMDA_con, self.NMDA_eff, _ = connectivity['NMDA']
```

Only AMPA plasticity is kept. GABA and NMDA plasticity matrices are discarded because no GABA or NMDA synapses are plastic.

LastConductanceNMDA tracks NMDA saturation: a 2D structure mirroring the connection matrices, recording the conductance level at each synapse's last spike.

4.5 Dopamine State

All populations get dopamine arrays, but only dSPN (`dpmn_type` = 1) and iSPN (`dpmn_type` = 2) are actually processed. For all other populations, `dpmn_type` = 0 and the plasticity code skips them.

Key dynamic variables:

- **dpmn_DAp**: Phasic dopamine. Set by reward, decays with $\tau = 2$ ms.
- **dpmn_APRE/APOST**: Running spike traces for STDP.
- **dpmn_E**: Eligibility trace. Bridges STDP timing to dopamine arrival.
- **dpmn_XPRE/XPOST**: Binary spike indicators. Reset each timestep.

4.6 Temporary Arrays and Firing Rate Buffer

Pre-allocated scratch pads (`cond`, `g_rb`, `g_adr`, `g_k`, `dv`, `tau_n`, `n_inif`, `Vaux`) avoid memory allocation every timestep. `cond` is the integer mask: 1 for neurons to update, 0 for neurons in refractory or above threshold.

The **rollingbuffer** is the circular firing rate buffer: `n_pops` × `300` array. Each timestep writes spike count into the current column, pointer advances and wraps.

get_firing_rates() computes: mean spikes per timestep ÷ N neurons ÷ dt × 1000 = Hz.

5 timestep.py — The Core Simulation

This is the heart of BioMind-BG. Every other file sets up data structures. This file makes neurons **fire**. It computes one $dt = 0.2$ ms step. Called 5000 times per second of brain time.

5.1 Step 1: Reset Spike Indicators

```
for i in range(n_pops):
    a.dpmn_XPRE[i] *= 0
    a.dpmn_XPOST[i] *= 0
```

Every timestep starts fresh. XPRE and XPOST are binary flags set during Steps 3 and 6 when spikes occur.

5.2 Step 2: External Input (Ornstein-Uhlenbeck)

```
a.ExtMuS_AMPA[i] = eff * freq * 0.001 * N_conn * tau
a.ExtSigmaS_AMPA[i] = eff * sqrt(tau * 0.5 * freq * 0.001 * N_conn)
a.ExtS_AMPA[i] += dt/tau * (-ExtS + Mu)
                  + Sigma * sqrt(2*dt/tau) * N(0,1)
a.LS_AMPA[i] *= exp(-dt / tau)
```

The external conductance follows a random walk pulled back toward its mean (Ornstein-Uhlenbeck process). Like a drunk person on a leash:

- First term: `dt/tau * (-ExtS + Mu)` pulls conductance toward the mean.
- Second term: Gaussian noise provides random kicks.
- Lateral conductance decays exponentially, waiting for new spikes in Step 3.

5.3 Step 3: Spike Propagation

For every neuron that spiked last timestep, add its synaptic weight to destination conductances:

AMPA: Direct increment + XPRE tracking for plasticity.

GABA: Direct increment only.

NMDA: Includes saturation mechanism. Each spike adds $\alpha \times (1 - \text{last_conductance})$ instead of a fixed amount, preventing unbounded accumulation during sustained firing.

This is the **slowest part of the code** due to the triple nested Python loop (populations × populations × spiking neurons).

5.4 Step 4: T-Current (Rebound Bursting)

```
cond = (a.V[i] < a.V_h[i]).astype(int)
a.h[i] += cond * dt * (1.0 - a.h[i]) / a.tauhp[i]        # charge
a.h[i] += (1 - cond) * dt * (-a.h[i]) / a.tauhm[i]        # decay
a.g_rb[i] = a.g_T[i] * a.h[i] * (1 - cond)                # burst
```

When hyperpolarized ($V < -60$ mV): h charges toward 1.0 (100 ms). When released: h drives a burst through g_T, then decays (20 ms). Only active in STN and GPe ($g_T = 0.06$). All other populations: $g_T = 0$, so $g_{rb} = 0$ always.

5.5 Step 5: Membrane Voltage Update

The main equation with five current terms:

$$\frac{dV}{dt} = -\left[\underbrace{\frac{V - V_{rest}}{\tau_m}}_{\text{leak}} + \underbrace{\frac{Ca \cdot g_{AHP}}{C}(V - V_K)}_{\text{afterhyperpol.}} + \underbrace{\frac{g_{ADR}}{C}(V - V_{ADR})}_{\text{delayed rect.}} + \underbrace{\frac{g_K}{C}(V - V_{ADR})}_{\text{outward K}^+} + \underbrace{\frac{g_{rb}}{C}(V - V_T)}_{\text{T-current}}\right] \tag{1}$$

Then three synaptic currents are added:

$$\Delta V_{\text{AMPA}} = dt \cdot \frac{(E_{\text{AMPA}} - V) \cdot (S^{lat}_{\text{AMPA}} + S^{ext}_{\text{AMPA}})}{C} \times 0.001 \tag{2}$$

$$\Delta V_{\text{GABA}} = dt \cdot \frac{(E_{\text{GABA}} - V) \cdot (S^{lat}_{\text{GABA}} + S^{ext}_{\text{GABA}})}{C} \times 0.001 \tag{3}$$

$$\Delta V_{\text{NMDA}} = dt \cdot \frac{(E_{\text{NMDA}} - V) \cdot (S^{lat}_{\text{NMDA}} + S^{ext}_{\text{NMDA}})}{C \cdot (1 + e^{-0.062V/3.57})} \times 0.001 \tag{4}$$

The $(E_{rev} - V)$ term is the driving force. For AMPA ($E = 0$ mV): at rest ($V = -70$), driving force $= +70$ (excitatory). For GABA ($E = -70$ mV): at threshold ($V = -50$), driving force $= -20$ (inhibitory). NMDA has the additional Mg^{2+} block denominator.

Neurons in refractory period (`cond = 0`) skip the entire update.

5.6 Step 6: Spike Detection

```
spiked = np.nonzero(a.V[i] > a.Threshold[i])[0]
for neuron in spiked:
    a.V[i][neuron] = 0.0            # spike marker
    a.Ca[i][neuron] += alpha_ca     # calcium influx
    a.RefrState[i][neuron] = 10     # 2ms refractory
    a.Ptimesincelastspike = a.timesincelastspike  # save
    a.timesincelastspike = 0.0      # reset timer
    a.dpmn_XPOST[i][neuron] = 1.0   # mark for plasticity
```

Six things happen per spike: voltage marked, calcium added, refractory started, timer saved then reset, post-synaptic indicator set.

5.7 Step 7: Dopamine Plasticity (Three-Factor Learning)

Only runs for populations with `dpmn_type > 0` (dSPN and iSPN).

Factor 1 — STDP Eligibility Trace:

$$\frac{dA_{pre}}{dt} = \frac{\delta_{pre} \cdot X_{pre} - A_{pre}}{\tau_{pre}} \tag{5}$$

$$\frac{dA_{post}}{dt} = \frac{\delta_{post} \cdot X_{post} - A_{post}}{\tau_{post}} \tag{6}$$

$$\frac{dE}{dt} = \frac{X_{post} \cdot A_{pre} - X_{pre} \cdot A_{post} - E}{\tau_E} \tag{7}$$

$E > 0$: pre-before-post (LTP direction). $E < 0$: post-before-pre (LTD direction). $\tau_E = 100$ ms holds this signal for dopamine arrival.

Factor 2 — Dopamine Signal:

$$DA = m \cdot (DA_{phasic} + DA_{tonic}) \tag{8}$$

Transformed through receptor-specific $f(DA)$:

- D1: linear with slope $y/x = 6.0$, clamps at $-y = -3.0$ below $DA = -0.5$
- D2: linear with slope $y/x \cdot \varepsilon = 1.8$, clamps at $y \cdot \varepsilon = 0.9$ above $DA = 0.5$

Factor 3 — Weight Update:

$$\Delta w = dt \cdot c_{ij} \cdot \alpha_w \cdot f(DA) \cdot E \cdot \begin{cases} (w_{max} - w) & \text{if } \Delta w > 0 \\ (w - 0.001) & \text{if } \Delta w < 0 \end{cases} \tag{9}$$

Soft multiplicative bounds: weights approach but never reach w_{max} or 0.001.

5.8 Step 8: Firing Rate Buffer

```
for i in range(n_pops):
    a.rollingbuffer[i][a.bufferpointer] = len(a.spikes[i])
a.bufferpointer = (a.bufferpointer + 1) % a.bufferlength
```

Write spike count, advance pointer, wrap at 300.

6 qlearning.py — Reward to Dopamine Bridge

This module bridges **behavior** and **neurons**. It translates "you chose action 0 and got +1 reward" into a dopamine signal that the spiking network can use to update its weights.

It implements the Rescorla-Wagner learning rule (1972), which is the simplest form of reinforcement learning. The prediction error maps to Schultz's (1997) RPE theory of dopamine: dopamine neurons fire when outcomes are **better** than expected, pause when **worse** than expected.

6.1 QLearner Class

```
class QLearner:
    def __init__(self, n_actions=2):
        self.alpha = 0.1        # learning rate
        self.C_scale = 80.0     # RPE amplification
        self.Q = np.full(n_actions, 0.5)  # initial Q-values
        self.Q_history = [self.Q.copy()]
        self.DA_history = []
```

alpha = 0.1: Q-values update by 10% of the prediction error per trial. Slow, stable learning.

C_scale = 80: Amplifies the abstract RPE (a number between −2 and +1) to neural scale. RPE of +0.5 becomes DA of +40.

Q starts at 0.5: No prior bias. Both actions equally valued.

6.2 update() — The Core Learning Step

Three things happen every trial:

1. **Prediction Error:** $RPE = reward - Q[chosen]$

   Example: $Q[0] = 0.5$, reward $= +1.0 \Rightarrow RPE = +0.5$ (better than expected)

2. **Q-Value Update:** $Q[chosen] \mathrel{+}= \alpha \cdot RPE$

   Example: $Q[0] = 0.5 + 0.1 \times 0.5 = 0.55$

3. **Dopamine Signal:** $DA = RPE \times C_{scale}$

   Example: $DA = 0.5 \times 80 = +40.0$ (injected into all striatal neurons)

7 trial.py — The Behavioral Task

Manages the three-phase trial loop that sits on top of the neural simulation.

7.1 Phase 0: Stimulus Presentation

Cortical input is turned on. Each action channel gets `FreqExt_AMPA = basestim + gain` × `max_stim`. The thalamic firing rate is monitored every millisecond (5 timesteps).

A **decision** is detected when any thalamic channel crosses 30 Hz. The first channel to cross wins (first-past-the-post). If 1000 ms pass with no crossing, it is a timeout (no choice).

The neural mechanism: stimulus drives cortex → cortex excites dSPN → dSPN inhibits GPi → GPi's tonic inhibition of thalamus weakens → thalamus firing rate rises → crosses 30 Hz → decision.

7.2 Phase 1: Movement Delay

After decision, gain on the unchosen action drops to 0, chosen action drops to 30%. Network waits 300 ms. During this delay, the eligibility trace ($\tau_E = 100$ ms) is decaying. Dopamine must arrive before it fades.

7.3 Phase 2: Reward and Inter-Trial Interval

The chosen action is evaluated against the reward schedule. Q-values update. Phasic dopamine is computed and injected into ALL striatal populations (both dSPN and iSPN, both channels). After 300 ms ITI, everything resets for the next trial.

7.4 Key Implementation Detail

```python
def step(self):
    # Apply stimulus to cortex
    # Run 5 neural timesteps (1 ms)
    from biomind.timestep import multi_timestep
    multi_timestep(a, 5)
    # Check for phase transitions
```

Each call to `step()` advances the simulation by 1 ms (5 × 0.2 ms). The trial manager calls this in a loop until all trials are complete.

8 run.py — Orchestration

Ties everything together in a simple sequence:

1. Build populations and connectivity (`populations.py`)
2. Initialize agent (`agent.py`)
3. Zero cortical stimulus and run 5000-step warmup (`timestep.py`)
4. Generate reward schedule (probabilistic, from seed)
5. Create QLearner and TrialManager
6. Loop: call `trial_mgr.step()` until all trials complete
7. Collect and return results

`run_baseline_test()` runs only the warmup and reports firing rates, useful for validating the network without running trials.

`run_simulation()` runs the full experiment with trials, learning, and behavioral output.

9 Validated Results

9.1 Experiment 1: Baseline Firing Rates

Population	Model (Hz)	Expected (Hz)	Status
Cortex	0.0	0–5	PASS
CxI	0.8	0–10	PASS
dSPN	3.8	0–8	PASS
iSPN	4.2	0–8	PASS
FSI	7.8	2–30	PASS
GPe	60.7	20–80	PASS
STN	24.9	8–40	PASS
GPi	71.0	30–100	PASS
Thalamus	6.8	0–20	PASS

9.2 Experiment 4: Lesion Studies

Condition	GPi (Hz)	Th (Hz)	Clinical Match
Healthy	71.0	6.8	Normal
Parkinson (D1 loss)	81.0	3.4	Bradykinesia (can't initiate)
Parkinson (D2 overactive)	85.4	2.2	Severe bradykinesia
Huntington (iSPN loss)	50.4	15.0	Hyperkinesia (involuntary moves)
STN lesion (DBS)	0.0	46.2	Therapeutic effect confirmed